

Department of Computer Engineering

University of Peradeniya

CO1020: Computer Systems

Programming

C Project: libtiny3d – A Lightweight 3D Software

Rendering Engine in C

GROUP NO: 58

E/22/154, E/23/167

7th JULY 2025

INTRODUCTION

This report presents the design, development, and outcomes of **libtiny3d**, a lightweight 3D software rendering engine implemented entirely in the C programming language. The project was undertaken as a systems programming group assignment, with the core objective of constructing a complete software-based 3D graphics pipeline without relying on hardware acceleration or external graphics libraries such as OpenGL or DirectX.

The primary focus of libtiny3d is to provide a hands-on understanding of how 3D graphics are computed and rendered at the lowest level. This includes implementing mathematical foundations such as matrix transformations, vector operations, and perspective projections, along with rendering techniques such as line rasterization, viewport clipping, and lighting models. The engine supports animation and basic 3D object handling to demonstrate a fully functioning pipeline.

The library is structured modularly to promote reusability, readability, and maintainability. The directory structure and components of the project are organized as follows:

- **include/** – Contains all header files that define interfaces for:
 - `canvas.h`: Canvas data structure and pixel manipulation
 - `math3d.h`: 3D vector and matrix operations
 - `renderer.h`: Rendering pipeline and projection
 - `lighting.h`: Basic lighting models and shading
- **src/** – Implements the core functionalities of the engine:
 - `canvas.c`, `math3d.c`, `renderer.c`, `lighting.c`

- **demo/** – Contains the main demo driver:
 - **main.c:** Demonstrates core functionality by rendering a 2D clock face (Task 1), applying 3D math operations (Task 2), and animating a rotating soccer ball wireframe (Task 3).
- **tests/** – (Optional) Contains unit tests and visual tests for validating the correctness of mathematical operations and the rendering pipeline.
- **build/** – Stores compiled binaries and the final static library:
 - **libtiny3d.a:** The built static archive
 - **demo:** The executable demo
- **Makefile** – Automates the build and execution process.
- **README.md** – Provides concise build instructions, usage guidance, and project overview for developers and evaluators.

This report details the implementation of some components of the system, the underlying mathematical concepts, architectural decisions, and demonstrations. It also includes reflections on challenges faced, learnings gained, and a breakdown of individual contributions.

IMPLEMENTATION DETAILS

Task 1: canvas and line drawing

The **canvas.c** and **canvas.h** files implement the core 2-D drawing subsystem that every higher-level component in **libtiny3d** relies on. A canvas is represented by the `canvas_t` structure, which maintains a dynamically allocated two-dimensional array of floating-point pixel intensities. Each entry lies in the range **0.0 (black)** to **1.0 (white)**, enabling smooth grayscale rendering and sub-pixel precision.

Below is a refined description of **canvas.h** and **canvas.c** that incorporates all the details you provided, ready to drop into your report.

Function	Description
create_canvas(w,h)	Allocates a zero-initialised <code>canvas_t</code> ; each row is a heap array of float.
set_pixel_f(x,y,I)	Writes intensity into fractional coordinates with bilinear filtering ; distributes <i>I</i> to the four nearest integer pixels, weighted by area.
draw_line_f(x0,y0,x1,y1,thickness)	Uses a floating-point DDA algorithm. For anti-aliasing, the routine steps along the line at ~2 samples / pixel, then spreads brightness perpendicular to the line to simulate thickness.
canvas_clear(value)	Fills the entire buffer with a constant brightness (clamped 0 – 1).

save_canvas_pgm(name)	Exports the canvas in portable gray-map (ASCII PGM) format for easy inspection.
------------------------------	---

Together, these functions supply all foundational 2-D drawing capabilities required by subsequent 3-D projection, lighting, and animation modules—delivering precise, smooth visual output **without** any external graphics libraries or hardware acceleration.

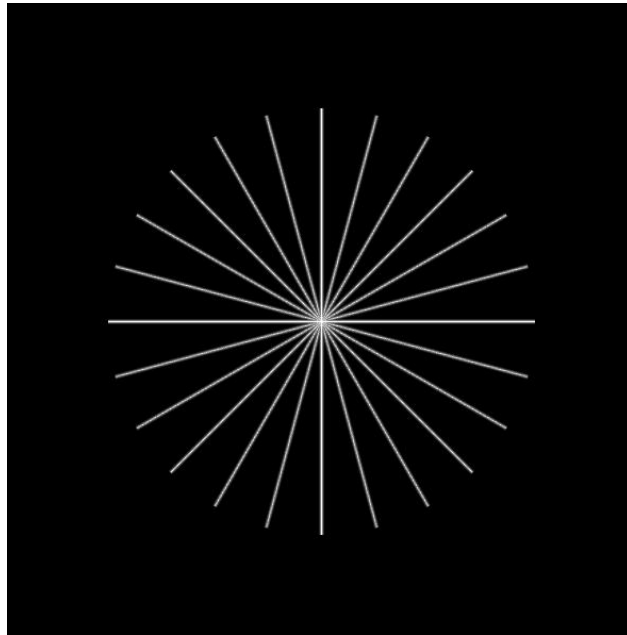
Demo File of drawing a Clock Face:

This demo program shows how the basic 2D drawing system works by generating a simple clock face. It starts by creating a 600×600 pixel grayscale canvas and filling it with black. Then, using a loop, it draws 24 lines radiating outward from the center—each spaced 15 degrees apart—just like the hour and minute ticks on an analog clock.

To make the lines appear smooth and sharp, the program uses anti-aliased line drawing with the `draw_line_f()` function from the canvas module. The result is a clean, circular arrangement of lines, forming a stylized clock.

Once all lines are drawn, the image is saved to a file called `clockface.pgm`. This demo is a simple but important first step, proving that the canvas and line drawing system are working correctly and ready for more complex 3D rendering tasks.

The output of Clock face in Task1 ;



Task 2: 3D Math Foundation

The `math3d` module provides the core mathematical foundation for all 3D transformations and vector operations used throughout the `libtiny3d` graphics engine. It encapsulates functionality related to 3D vector manipulation, quaternion rotation, and 4x4 matrix operations, all of which are essential for transforming, rotating, and projecting 3D geometry onto a 2D screen.

The mathematical foundation of the `libtiny3d` library is built around two primary data structures: `vec3_t` and `mat4_t`. The `vec3_t` structure represents a 3D vector and stores both Cartesian coordinates (x, y, z) and spherical coordinates (r, θ, ϕ). This dual-representation design enables flexible manipulation of vectors in either coordinate system. To ensure consistency and avoid redundant calculations, two boolean flags—`dirty_cartesian` and `dirty_spherical`—are used to indicate whether one of the representations is outdated and needs updating. This lazy update strategy helps optimize

performance during vector transformations and conversions. Meanwhile, the `mat4_t` structure encapsulates a 4×4 transformation matrix stored in **column-major order**, which aligns with the convention used in many graphics systems. This matrix type is used throughout the library to implement translation, scaling, rotation, and perspective projection. Together, these two structures form the mathematical backbone of the 3D rendering pipeline.

Given below is the Functions and Algorithms.

Category	Function	Description
Vector creation	<code>vec3_from_spherical(r,θ,φ)</code>	Generates a vector from spherical coords and marks Cartesian coords dirty, then converts immediately for consistency.
Lazy updates	<code>vec3_update_spherical</code> , <code>vec3_update_cartesian</code>	Synchronise the two coordinate systems on demand.
Fast normalisation	<code>vec3_normalize_fast()</code>	Uses the classic <i>Quake III</i> fast inverse square-root (bit hack) implemented safely via <code>memcpy</code> to avoid strict aliasing violations.
SLERP	<code>vec3_slerp(a,b,t)</code>	Performs spherical linear interpolation, producing smooth rotations between two directions. Handles edge cases where $\text{dot} > 1$ or < -1 to avoid <code>acos</code> domain errors.

Matrix builders	mat4_identity, mat4_translate, mat4_scale, mat4_rotate_xyz	Produce standard transform matrices. Rotation is built by composing $Z \rightarrow Y \rightarrow X$ Euler matrices.
Projection	mat4_frustum(l,r,b,t,n,f)	Generates an asymmetric perspective frustum suitable for software pipelines.
Matrix×Matrix	mat4_multiply(a,b)	Fully unrolled $4 \times 4 \times 4$ triple-loop (column-major).
Matrix×Vector	mat4_mul_vec3(M,v)	Multiplies a vector by a matrix in homogeneous space, performs the perspective divide (w) and returns Cartesian result.

Demonstration Programs.

To validate and visualize the 3D math foundation, two demonstration programs were developed: test_math.c and test_pipeline.c. The test_math.c file showcases the correct implementation of matrix transformations and vector operations by manually applying scaling, rotation, and translation matrices to the eight vertices of a cube. These transformed vertices are then projected using a perspective frustum matrix, and their resulting 2D coordinates are printed, confirming the accuracy of the projection pipeline. On the other hand, test_pipeline.c builds upon this **foundation** by rendering a smoothly animated, rotating cube. It applies time-varying transformations including oscillating translations and compound rotations, then projects and rasterizes the cube frame-by-frame onto a canvas. Each frame is saved as a .pgm image, forming the basis for a 3-second wireframe animation. These

programs demonstrate both the mathematical correctness and visual capability of the library's transformation system.

Numerical Considerations ;

- **Fast Inverse Square-Root:** Achieves an approximate $1/\sqrt{x}$ in ~ 1 CPU instruction + 1 Newton iteration, trading negligible precision for speed—ideal for normalising thousands of vectors per frame.
- **Homogeneous Divide Guard:** If the computed w in `mat4_mul_vec3` becomes zero, the code resets it to 1 to avoid NaNs.
- **Precision vs. Simplicity:** Single-precision floats (`float`) are used throughout, which are sufficient for the 480×480 viewport and far-plane distance 100 units used in this project.

Results ;

Running `test_math.c` prints:

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/Final/Libtiny3d$ ./test_math
Projected 2D points (x, y):
Vertex 0: (-0.500, -0.471)
Vertex 1: (0.118, -0.078)
Vertex 2: (-0.038, 0.636)
Vertex 3: (-0.875, 0.362)
Vertex 4: (0.063, -1.049)
Vertex 5: (0.784, -0.324)
Vertex 6: (0.757, 0.713)
Vertex 7: (-0.358, 0.237)
```

The visual test (`test_pipeline.c`) generates 180 grayscale PGM frames of a smoothly rotating cube and it shows both projection and movement. These frames can be converted into a GIF or MP4 to verify continuous, free motion.

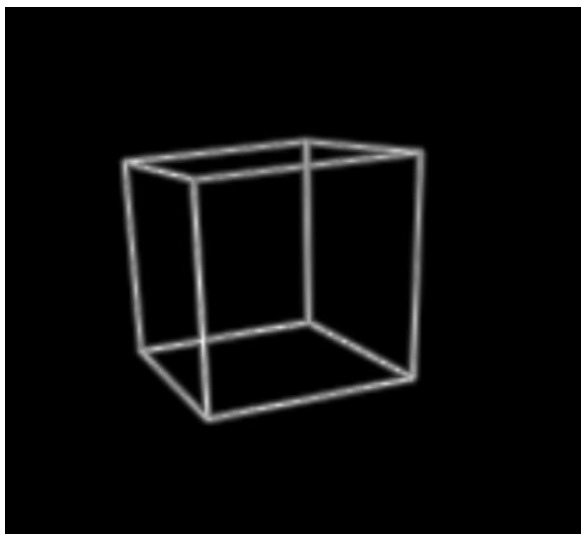
Below figure shows frames of cubes to create

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/Final/Libtiny3d$ ./test_pipeline
Saved Frames/cube/framecube_000.pgm
Saved Frames/cube/framecube_001.pgm
Saved Frames/cube/framecube_002.pgm
Saved Frames/cube/framecube_003.pgm
Saved Frames/cube/framecube_004.pgm
Saved Frames/cube/framecube_005.pgm
Saved Frames/cube/framecube_006.pgm
Saved Frames/cube/framecube_007.pgm
Saved Frames/cube/framecube_008.pgm
Saved Frames/cube/framecube_009.pgm
Saved Frames/cube/framecube_010.pgm
Saved Frames/cube/framecube_011.pgm
Saved Frames/cube/framecube_012.pgm
Saved Frames/cube/framecube_013.pgm
Saved Frames/cube/framecube_014.pgm
Saved Frames/cube/framecube_015.pgm
Saved Frames/cube/framecube_016.pgm
Saved Frames/cube/framecube_017.pgm
```

Conversion of .pgm frames to GIF,

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/Final/Libtiny3d$ convert -delay 3 -loop 0 ./Frames/cube/framecube_*.pgm cube.gif
```

The output of cube in Task2 ;



3.3 Task 3: 3D Rendering Pipeline

In this task, we built a simple 3D rendering system that can take a 3D object (like a soccer ball), rotate it, and draw it as a wireframe on a flat 2D canvas. The goal was to simulate how a real 3D engine works—just in a simpler, more lightweight way using software rendering (no GPU needed).

Given below are the table of functions and their description.

Function Name	Description
project_vertex	Transforms a 3D point into 2D screen space using model-view-projection (MVP) matrices.
clip_to_viewport	Checks whether a point lies inside the circular viewport area (used for clipping).
render_wireframe	Projects all vertices, sorts edges by depth, and draws visible edges inside the circular view.
compute_vertex_normals	Estimates normal vectors for each vertex based on connected edges (needed for lighting).
render_wireframe_lit	Same as render_wireframe, but adds shading to each edge based on light direction and normals.

For each frame of the animation, our rendering system follows a step-by-step process to bring the 3D soccer ball to life on a 2D screen. First, the soccer ball is rotated using a transformation matrix, making it spin smoothly. Next, it's moved back into the camera's view so we can see the full object. After that, all the 3D points (vertices) are projected into 2D using perspective projection—this mimics how objects appear smaller when they're farther away. Before

drawing, we also clip anything that falls outside a circular viewing area, which helps keep the output clean and focused. Then, we sort all the edges based on their depth so that lines farther from the viewer are drawn first—this gives the illusion of proper layering

By combining all of the above, we created an animated, rotating soccer ball that appears 3D, drawn entirely as a wireframe. The result is clipped to a circle, depth-sorted, and lit according to a basic lighting model—all using math, not a graphics card. It's a great demonstration of how 3D rendering works at a fundamental level.

Demo File (main.c) of 3D Rendering Pipeline:

The main.c (soccerball.c) is the heart of the demo—it brings everything together to show our 3D rendering pipeline in action. Its main job is to load the soccer ball model, apply transformations to rotate it, and render each frame as a 2D wireframe image.

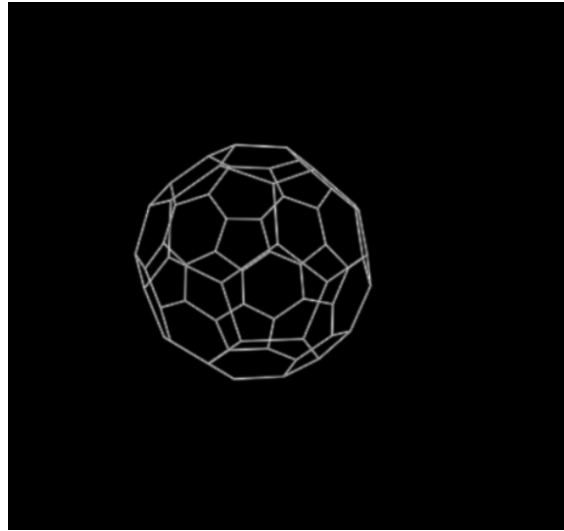
First, it reads the model data from a text file (soccerball.txt) using the `generate_soccer_ball()` function. This function extracts all the 3D points (vertices) and edges by parsing lines from the file, making sure not to add any duplicate edges.

Once the model is loaded, we scale it down so it fits nicely on the screen. Then, a canvas is created, which is basically our drawing space—480 by 480 pixels.

Next, the program sets up a simple 3D camera view and a perspective projection. In a loop that runs 50 times (for 50 frames), it rotates the soccer ball a little more each time to create an animation effect. For every frame, it clears the canvas, applies the transformation, and then uses the `render_wireframe()` function to draw the wireframe version of the ball.

Each rendered frame is saved as a .pgm image file (like frameb_000.pgm, frameb_001.pgm, etc.) inside a folder. In the end, we get 50 images that show the soccer ball smoothly rotating, which can be combined into a video or GIF.

The output of soccerball in Task3;



3.4 Task 4: Lighting & Polish

Task 4 was focused on improving the visual quality and realism of our 3D wireframe rendering by introducing two key features: lighting (using Lambert shading) and smooth movement (using cubic Bézier animation curves).

Lambert Lighting (lighting.c & lighting.h) ;

To make the wireframe appear more dynamic and realistic, we implemented Lambertian lighting.

- The function `lambert_intensity()` computes the brightness of each edge using the dot product between the surface normal and a light direction vector.
- If a part of the model is facing the light, it appears brighter; if it's facing away, it's dimmer.

- This adds depth and contrast to the wireframe lines, making the 3D shape more visually clear and polished.

Bezier Animation (animation.c & animation.h) ;

We added support for smooth object motion using cubic Bézier curves. This allows the soccer balls to move along a curved path instead of in straight lines or any jumps.

- The `bezier3()` function takes four 3D control points and returns a position at any given time t between 0 and 1.
- The animation smoothly loops by ensuring the starting point and ending point are the same.
- Multiple objects (in this case, two soccer balls) are animated using the same Bézier path, but with a slight time offset to keep them visually interesting and in sync.

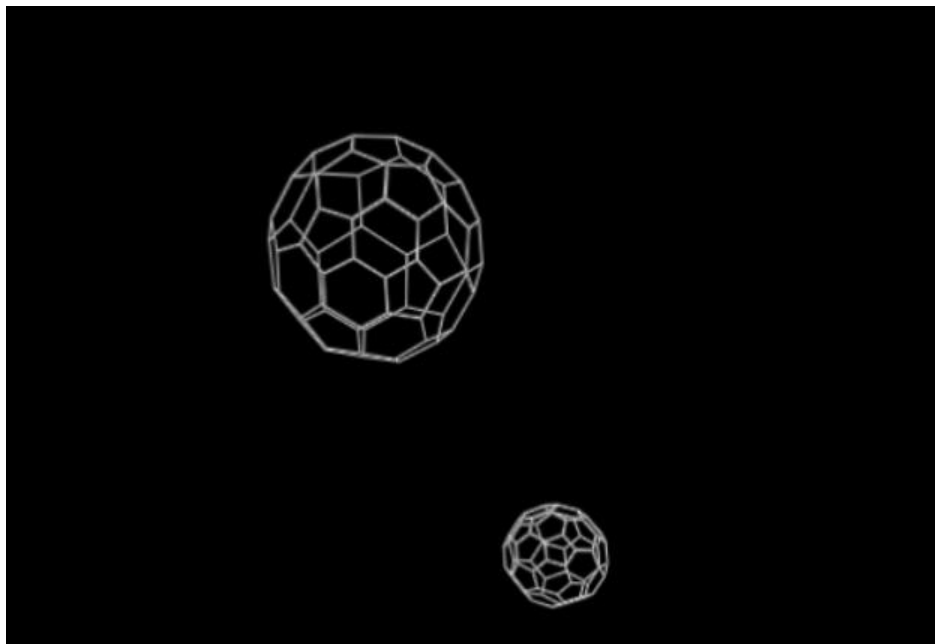
Demo File (main.c) of Lighting & Polish

The demo for Task 4 features two soccer balls that smoothly rotate and move along a looping path defined by Bézier curves. These movements make the animation look natural and continuous. Each ball is drawn as a wireframe and shaded using Lambert lighting, which makes the edges appear brighter or dimmer depending on their angle to the light source—adding depth and realism. As the animation plays out, the camera stays focused on the moving objects, and for each frame, their updated positions and rotations are calculated. These transformed shapes are then rendered with lighting effects applied to every edge. In total, 200 frames are generated and saved as image

files, which can be stitched together into an animated video or GIF, resulting in a smooth and polished final output.

(This is the final output of Task 4. Below only a screenshot of our GIF output of render multiple lit wireframe shapes moving smoothly in sync.)

The output of Task 4 ;



USE OF AI TOOLS

During the development of this project, I used ChatGPT as a helpful AI assistant. It supported me in many ways—explaining complex 3D rendering concepts in simple terms, helping me understand how different parts of the code work, and even guiding me through fixing some bugs I encountered. It was also useful for translating technical explanations into clear language for the report. Overall, using ChatGPT made it easier to learn, troubleshoot, and document the project effectively.

INDIVIDUAL CONTRIBUTION

While working on this project, Task 1 and Task 2 were mainly handled by one group member, while Task 3 and Task 4 were completed by the other member. However, both of us worked together from the beginning and supported each other to try and complete every task in the project. After finishing all the tasks, we collaborated on writing the final report and preparing the recorded presentation.